DEVELOPER REFERENCE

Generated 12 October 2026 · package version v1.7.1

# Using `peoplelogy/laravel-xero-bridge` — Xero

A practical reference for calling Xero through the package: how to reach it, which organisation you are talking to, what comes back, the contact flows, the events and commands, and what goes wrong.

This is one of the guides in this folder:

| Guide | Covers |
|---|---|
| **Xero usage** (this one) | The calling mechanism, connections, contacts, events, commands, error handling |
| **Invoice usage** | Creating, paying, sending, finding and listing invoices |
| **MyInvois usage** | Validating a Malaysian taxpayer TIN with LHDN, and recording the checks |
| **MyInvois user manual** | For finance and admin staff, not developers: running a TIN check from the console, step by step |

It covers the package only. Nothing here assumes a particular application, database or domain model — the package deliberately contains no bookkeeping logic, so *when* to raise an invoice and *what* goes on it stay yours to decide.

Every flow can be run by hand, against a real Xero organisation, from the package's own test console at `/xero/console`. It is **off until** `XERO_CONSOLE_ENABLED=true` is set in the environment that should have it — in every environment, whatever `APP_ENV` says — and its actions map one-to-one onto the flows in these guides, so the quickest way to understand a flow is usually to switch it on locally, run it there and read the JSON it returns (credentials and bank details are masked). Writes are refused unless the connected organisation is a Xero Demo Company or explicitly allow-listed. Reads are not: behind the default `web,auth` middleware any signed-in user can read that organisation's invoices and contacts, so narrow `XERO_CONSOLE_MIDDLEWARE` wherever you turn it on.

> **Version note.** Written against `peoplelogy/laravel-xero-bridge` v1.7.1. Method signatures are read from the package source; request and response samples are reproduced from the package's numbered docs in the folder above this one, which are the authority wherever the two disagree. The GUIDs and figures in them are illustrative.

## Contents


1. The calling mechanism
2. Which organisation you are talking to

3. What you get back
4. Contacts — first or create
5. Contacts — find one
6. Events you can listen to
7. Artisan commands
8. Error handling
9. Related

---

## The calling mechanism

### The namespace

The package autoloads PSR-4 from a single root:

```
Peoplelogy\XeroBridge\   →   vendor/peoplelogy/laravel-xero-bridge/src/
```

So every class below is `Peoplelogy\XeroBridge\` plus its path — `Filters\InvoiceFilter` is `Peoplelogy\XeroBridge\Filters\InvoiceFilter`, and so on. The service provider is auto-discovered; there is nothing to register in `bootstrap/providers.php`.

### Three ways to reach it

**1. Import the facade — the usual choice.**

```
namespace App\Services\Xero;

use Peoplelogy\XeroBridge\Facades\XeroBridge;

class ContactSync
{
    public function find(string $email): ?array
    {
        return XeroBridge::contacts()->findByEmail($email);
    }
}
```

**2. The global alias**, registered by the package as `XeroBridge`. No import, but it lives in the **root** namespace:

```
\XeroBridge::contacts()->findByEmail($email);   // note the leading backslash
```

> **The mistake everybody makes once.** Inside a namespaced class, writing `XeroBridge::contacts()` with no `use` and no leading `\` resolves to `App\Services\Xero\XeroBridge` — a class that does not exist — and fatals with *"Class not found"* that names your own namespace, not the package's. Either add the `use` statement or write `\XeroBridge`. In Blade and `routes/*.php` (no namespace) the bare `XeroBridge::` works, which is what makes this inconsistent enough to trip over.

**3. Inject the manager** — no facade, and trivially mockable in tests:

```
use Peoplelogy\XeroBridge\XeroBridgeManager;

public function __construct(private readonly XeroBridgeManager $xero) {}

// then
$this->xero->contacts()->findByEmail($email);
```

`XeroBridgeManager` is a container singleton, also aliased as `'xero-bridge'`, so `app('xero-bridge')` and `app(XeroBridgeManager::class)` are the same object. The facade is a thin wrapper over exactly this — every method in these guides works identically through all three.

## Four resources

```
XeroBridge::contacts()     // Contacts
XeroBridge::invoices()     // Invoices — see the Invoice usage guide
XeroBridge::payments()     // Payments
XeroBridge::settings()     // Organisation, Accounts, TaxRates, BrandingThemes (read-only)
```

Anything the package has not wrapped is still reachable:

```
XeroBridge::request('GET', 'CreditNotes');          // decoded array
XeroBridge::raw('GET', 'CreditNotes');              // the undecoded Response: status,
headers, body
XeroBridge::client()->get('Currencies', ['where' => '...']);
```

All three send `Accept: application/json`, and a header of your own cannot override it — so for an invoice's PDF use `invoices()->pdf($id)`, which asks for one.

## What you import, and when

Only the facade is needed for ordinary use. The rest you import when a specific job calls for it:

| Import | When |
| --- | --- |
| `Facades\XeroBridge` | Always — the entry point |
| `Filters\InvoiceFilter` | Building an invoice `list()` / `all()` query (Invoice usage) |
| `Exceptions\*` | Catching a specific failure (error handling) |
| `Support\InvoiceTransitions` | Checking a status change is legal before attempting it |
| `Support\BatchResult` | Type-hinting the return of `createMany()` |
| `Models\XeroConnection` | Reading the stored connection row directly |
| `Events\*` | Writing a listener (events) |
| `OAuth\Actor` | Type-hinting `XeroConnected::$actor`, or building one in a test |
| `Contracts\ConnectionRepository` | Replacing how connections are stored |
| `Webhooks\WebhookSignature` | Signing a test webhook delivery: `compute()`, `isValid()`, `HEADER` |
| `Jobs\ProcessXeroWebhook` | Running your listeners in a test: `(new ProcessXeroWebhook($payload))->handle()` |
| `Models\XeroApiCall` | Reading captured API calls, through its `forOwner()`, `failed()`, `logicalCall()` and `channel()` scopes |
| `Capture\ApiCallRecorder` | Attributing captured calls to one of your records, with `forOwner()` |
| `MyInvois\*` | Validating a TIN and reading its records (MyInvois usage) |

Everything else under `Client\`, `OAuth\`, `Http\`, `Jobs\`, `Repositories\` and `Webhooks\` is internal plumbing — apart from `OAuth\Actor`, `Webhooks\WebhookSignature` and `Jobs\ProcessXeroWebhook` above, and the `Webhooks\WebhookEvent` and `WebhookEnvelope` objects that `XeroWebhookReceived` hands you. The README's Versioning section lists the whole public API. If you find yourself importing from there, check the facade cannot already do it.

### What it does for you

Token refresh (access tokens last 30 minutes, refresh tokens rotate and are refreshed under a per-connection lock), retries with backoff, `Retry-After` handling, `Idempotency-Key` on writes, and typed exceptions. You do not write any of that.

### Dates

Send plain `Y-m-d` strings. Xero **returns** `/Date(1772409600000+0000)/`, which is not JSON-parseable as a date — read `DateString` / `DueDateString` instead, or convert the milliseconds yourself with `Support\XeroDate`. A date read back from Xero and re-posted unchanged will be rejected.

---

## Which organisation you are talking to

Most applications connect exactly one organisation, in which case you never name it:

```
XeroBridge::contacts()->findByEmail('finance@acme.com');                    // the default
connection
XeroBridge::connection('acme')->contacts()->findByEmail('finance@acme.com');  // a named one
```

`connection()` returns a **clone**, not `$this`. `XeroBridge::connection('acme')->contacts()` is scoped to that expression alone and cannot leak into the next call in the same request. The default connection name comes from `XERO_DEFAULT_CONNECTION` (`default`).

An organisation is connected by sending an administrator to **`/xero/connect`** (or `/xero/connect/{key}` for a named connection). The routes sit behind `web,auth` by default — any signed-in user can connect, or repoint an existing connection — so put your own gate in `XERO_ROUTES_MIDDLEWARE`.

---

## What you get back

Records come back as **plain PHP arrays, already unwrapped** from Xero's envelope. Xero answers `POST /Invoices` with `{"Invoices": [ ... ]}`; `create()` hands you the inner element.

| Shape | Methods |
|---|---|
| `array` — the record | `create()`, `update()`, `authorise()`, `organisation()` |
| `?array` — `null` when absent, no exception | `find()`, `findByEmail()` |
| `list<array>` — possibly empty | `list()`, `accounts()`, `taxRates()` |
| `LazyCollection` of records | `all()` — walks every page |
| `BatchResult` — per-invoice outcome | `createMany()` |
| `void` | `email()` — Xero answers 204 with no body |
| `string` | `pdf()` (raw bytes), `onlineUrl()` |

---

## Contacts — first or create

```
$contact = XeroBridge::contacts()->firstOrCreate(
    ['EmailAddress' => 'finance@acme.com'],   // exactly ONE lookup key
    [                                          // merged in only when creating
        'Name' => 'Acme Sdn Bhd',
        'FirstName' => 'Siti',
        'LastName' => 'Rahman',
        'CompanyNumber' => '202401012345',     // BRN
        'TaxNumber' => 'C12345678901',         // TIN
    ],
);
```

### Parameters

| Argument | Type | Notes |
|---|---|---|
| `$lookup` | `array<string,string>` | **Exactly one** key, either `EmailAddress` or `Name`. Anything else throws `InvalidArgumentException`. |
| `$attributes` | `array<string,mixed>` | Applied **only on create**. An existing contact comes back untouched. |

### On the wire

A lookup first (`GET /Contacts?where=EmailAddress=="finance@acme.com"`), then, only if nothing matched, a create:

```
PUT /api.xro/2.0/Contacts
{
  "Contacts": [
    {
      "EmailAddress": "finance@acme.com",
      "Name": "Acme Sdn Bhd",
      "CompanyNumber": "202401012345",
      "TaxNumber": "C12345678901"
    }
  ]
}
```

### Response


```
{
  "ContactID": "8138a266-fb42-49b2-a104-014b7045753d",
  "ContactStatus": "ACTIVE",
  "Name": "Acme Sdn Bhd",
  "EmailAddress": "finance@acme.com",
  "CompanyNumber": "202401012345",
  "TaxNumber": "C12345678901",
  "ContactPersons": [],
  "IsCustomer": true,
  "UpdatedDateUTC": "/Date(1790553726193+0000)/",
  "HasValidationErrors": false
}
```


Persist `ContactID`. It is the only stable way to reference a contact.

### Gotchas

- **It is a `PUT`, not a `POST`, on purpose.** `POST /Contacts` is an implicit upsert: matched on a non-unique name, it silently overwrites a real customer record. Xero's own guidance is to reference contacts by `ContactID`.
- **Race-safe.** If another process creates the contact between the lookup and the create, the loser gets a clean conflict and re-reads, rather than clobbering the winner.
- **Never use it to update.** An existing contact ignores `$attributes` entirely. To change a contact, call `contacts()->update($contactId, [...])`.
- `CompanyNumber` (BRN) and `TaxNumber` (TIN) are capped at 50 characters by Xero. Check the pair with LHDN before you write it — see the MyInvois usage guide.

---

## Contacts — find one

### By email — `contacts.find_by_email`

```
$contact = XeroBridge::contacts()->findByEmail('finance@acme.com');   // ?array
```

`GET /Contacts?where=EmailAddress=="finance@acme.com"`. Returns the **first** match or `null`.

**Gotcha:** Xero allows more than one contact per email address. Treat this as *a* match, not *the* match. `findAllByEmail()` returns every one if that distinction matters.

### By ContactID — `contacts.find`

```
$contact = XeroBridge::contacts()->find('8138a266-fb42-49b2-a104-014b7045753d');   // ?array
```

`GET /Contacts/{ContactID}`. The only unique way to reference a contact — Xero explicitly warns that contact names are not unique and recommends `ContactID` everywhere.

Both return the shape shown under first or create.

---

## Events you can listen to

All under `Peoplelogy\XeroBridge\Events\`, with public properties — every one readonly except `XeroConnected::$actor`:

| Event | Properties |
|---|---|
| `XeroConnected` | `XeroConnection $connection`, `bool $wasRepointed`, `?Actor $actor` |
| `TokenRefreshed` | `XeroConnection $connection`, `?CarbonImmutable $expiresAt` |
| `ConnectionExpired` | `XeroConnection $connection`, `string $reason`, `string $connectUrl` |
| `InvoiceCreated` | `string $connectionKey`, `array $invoice`, `?string $idempotencyKey` |
| `XeroWebhookReceived` | `WebhookEvent $event`, `WebhookEnvelope $envelope` — plus `connection()`, the stored `?XeroConnection` for the event's organisation |
| `XeroWebhookSignatureFailed` | `?string $signature`, `int $bodyLength`, `?string $ip` |
| `XeroWriteBlocked` | `string $connectionKey`, `string $operation`, `?string $ownerType`, `?string $ownerId`, `?string $reference`, `bool $pending`, `?string $xeroId`, `?CarbonImmutable $claimedAt` |

```
use Peoplelogy\XeroBridge\Events\ConnectionExpired;

Event::listen(ConnectionExpired::class, function (ConnectionExpired $e) {
    Notification::route('mail', config('app.admin_email'))
        ->notify(new XeroNeedsReconnecting($e->reason, $e->connectUrl));
});
```

`ConnectionExpired` is the one worth wiring up: it fires when the refresh token dies, which no amount of retrying fixes — a human has to reconnect. It does not fire when someone disconnects the organisation from inside Xero: the package cannot see that happen, and Xero typically answers the next call for it with a 403 (`XeroAuthenticationException`).

`XeroConnected::$actor` says who completed the consent: the signed-in user's `id`, `type` (the morph alias, or the class name) and `guard`, as plain scalars — or `null` when nobody was signed in on the callback. Read it as `$event->actor?->id`.

**Webhooks are opt-in.** `XeroWebhookReceived` and `XeroWebhookSignatureFailed` only ever fire when `XERO_WEBHOOK_KEY` is set: without a key the package registers no webhook route at all, so an application

that only calls Xero needs no webhook line in its environment file. With a key set, build `route:cache` where the key is present, or Xero gets a 404 — `xero-bridge:status` warns about exactly that. `XeroWebhookSignatureFailed` fires during Xero's intent-to-receive check by design, so alert on a sustained stream, not on one.

`XeroWriteBlocked` fires just before every `XeroWriteAlreadyClaimedException`. A block with `$pending` set and a `$claimedAt` more than a few minutes old is a stuck claim — something was sent to Xero and the outcome never recorded, and every later write for that record is refused until a person resolves it. That is the one to alert on.

---

## Artisan commands

```
php artisan xero-bridge:install          # publish config + migrations, print the .env keys
php artisan xero-bridge:status           # connections, expiry, config, tables, locks — no
API call
php artisan xero-bridge:refresh-tokens   # run hourly from your scheduler
php artisan xero-bridge:prune            # trim the optional tables, report stuck write
claims
```

```
// routes/console.php
Schedule::command('xero-bridge:refresh-tokens')->hourly()->withoutOverlapping()-
>onOneServer();
Schedule::command('xero-bridge:prune')->daily();   // nothing to do while the optional
tables are off
```

`xero-bridge:status` exits `0` when healthy, `1` on incomplete config, a connections table that is missing, unreadable or not the package's own, or stored tokens the current `APP_KEY` cannot decrypt, `2` when a connection needs reauthorising — and with `--strict`, `1` on any warning or a lock pre-flight that did not pass — so it can be wired straight into monitoring. It reports on Xero only; the MyInvois module is never part of it.

`xero-bridge:prune` covers every optional table the package owns — the write ledger, webhook replay records, captured API calls, and the MyInvois verdict and check-history tables — and skips any that were never migrated.

---

## Error handling

Every failure from Xero, and every refusal by one of the package's own guards, is a typed exception extending `Peoplelogy\XeroBridge\Exceptions\XeroBridgeException`. Catch the specific one when you can act on it differently; catch the base class as a backstop. A malformed argument — a second lookup

key, a page size out of range, a payment with no account — throws PHP's own `InvalidArgumentException` instead, before anything is sent.

The MyInvois module is the exception to the rule: it throws its own `MyInvoisException`, which is deliberately **not** a `XeroBridgeException` (see the MyInvois usage guide).

```
use Peoplelogy\XeroBridge\Exceptions\XeroBridgeException;
use Peoplelogy\XeroBridge\Exceptions\XeroRateLimitException;
use Peoplelogy\XeroBridge\Exceptions\XeroValidationException;

try {
    $contact = XeroBridge::contacts()->update($contactId, $changes);
} catch (XeroValidationException $e) {
    // Xero rejected the payload. $e->validationErrors() holds its own messages.
    report($e);
} catch (XeroRateLimitException $e) {
    // $e->retryAfter() is seconds, or null on a concurrency limit.
    $this->release($e->retryAfter() ?? 60);
} catch (XeroBridgeException $e) {
    report($e);
}
```

### The exceptions, in full

All under `Peoplelogy\XeroBridge\Exceptions\`, all extending `XeroBridgeException`:

`XeroBridgeException` · `XeroAuthenticationException` · `XeroConfigurationException` · `XeroConnectionNotFoundException` · `XeroReauthorizationRequiredException` · `XeroScopeException` · `XeroValidationException` · `XeroRequestException` · `XeroRateLimitException` · `XeroServiceUnavailableException` · `XeroIdentityUnavailableException` · `InvalidInvoicePayloadException` · `InvalidInvoiceTransitionException` · `InvoiceCannotBeVoidedException` · `UnsafeContactPayloadException` · `ConnectionKeyConflictException` · `TenantAlreadyConnectedException` · `XeroWriteAlreadyClaimedException` · `XeroWriteLedgerUnavailableException`

## What to catch, and what it means

| Exception | Cause | What to do |
|---|---|---|
| `XeroConfigurationException` | A `XERO_*` key is missing or wrong (bad redirect URI, no `offline_access`) | Fix the environment file, `php artisan config:clear`. Never retried. |
| `XeroConnectionNotFoundException` | Nothing connected under that key | Send an admin to `/xero/connect` |
| `XeroReauthorizationRequiredException` | The refresh token is dead (terminal `invalid_grant`) | Reconnect. No code change helps. |
| `XeroAuthenticationException` | Xero refused the token — typically a 403 after the organisation was disconnected inside Xero | Reconnect |
| `XeroScopeException` | The connection was authorised without a scope this call needs | Add it to `XERO_SCOPES` **and reconnect** — an existing connection never gains scopes |
| `XeroValidationException` | 400 from Xero: unknown `AccountCode`, missing contact, bad `TaxType` | Fix the payload. `validationErrors()` names the element. |
| `UnsafeContactPayloadException` | An invoice `Contact` block carries `ContactID` **plus** other fields | Send `ContactID` alone (Invoice usage) |
| `InvalidInvoicePayloadException`, `InvalidInvoiceTransitionException` | An invoice payload or status change the package refused before sending | See the Invoice usage guide |
| `XeroWriteAlreadyClaimedException` | The write ledger already holds this write (`for($order)`) | **Stop — do not retry.** `xeroId()` is the id already created; `isPending()` means something was sent and never recorded, so check Xero |
| `XeroWriteLedgerUnavailableException` | `XERO_WRITES_STRICT` is on and the ledger could not record the claim | Nothing was sent. **Retry** once the ledger table is migrated and the database is reachable |
| `XeroRateLimitException` | 429 | `retryAfter()` on minute/daily limits; **null on a concurrency limit** — back off yourself |
| `XeroServiceUnavailableException` | Organisation offline, or Xero is down | Retry in a few minutes |
| `XeroRequestException` | Anything else non-2xx | Inspect `statusCode()` and `response()` |

### Useful accessors on any `XeroBridgeException`

```
$e->statusCode();        // ?int
$e->connectionKey();     // ?string
$e->validationErrors();  // array — Xero's own messages
$e->xeroType();          // ?string
$e->xeroErrorNumber();   // ?int
$e->retryAfter();        // ?int seconds
$e->response();          // ?Illuminate\Http\Client\Response
$e->context();           // array
```

### In a queued job

`retryAfter()` exists so a worker can `release()` itself instead of sleeping. A daily rate limit can carry a `Retry-After` measured in hours — never sleep inline on one.

---

## Related

- **Invoice usage** and **MyInvois usage** — the other two guides in this folder.
- The numbered guides in `docs/` — `01-getting-started`, `02-invoices`, `03-contacts-payments-settings`, `04-webhooks-and-events`, `05-commands-and-errors`, `06-recipes`, `07-test-console`, `08-myinvois-tin-validation`, `09-persistence` and `10-api-capture`. Deeper than these guides on every topic, and the authority where the two disagree.
- The test console at `/xero/console` — once switched on with `XERO_CONSOLE_ENABLED=true`, runs every flow by hand and shows the request timings, the rate limit Xero reported and Xero's response, with credentials and bank details masked.
- `php artisan xero-bridge:status` — connections, token expiry, missing configuration, the package's tables and the lock store, without calling Xero.
- Xero Accounting API reference — for the fields this package passes through untouched.